Housing Data API for UK Rental and Sales Analysis

A RESTful API for UK housing data with lookup, analytics, authentication, and testing support

Shirui Zhao

2026-03-12

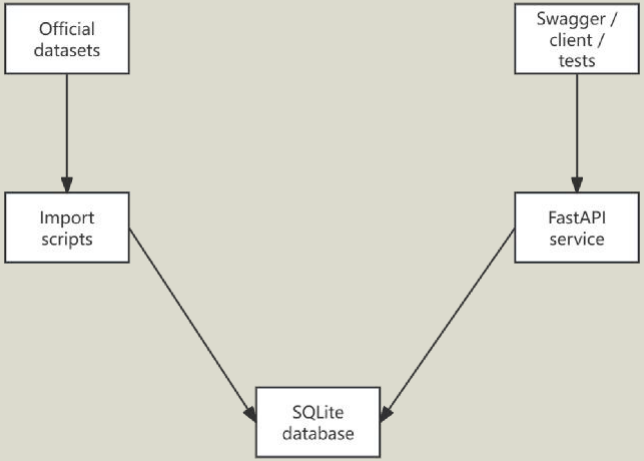
Problem and Motivation

- UK housing data is spread across multiple sources
- Raw datasets are not easy to query directly
- Users need a clean API for lookup, filtering, and trend analysis

- Build a RESTful, data-driven API
- Support both official data and user-managed records
- Emphasise good API design, database integration, testing, and documentation

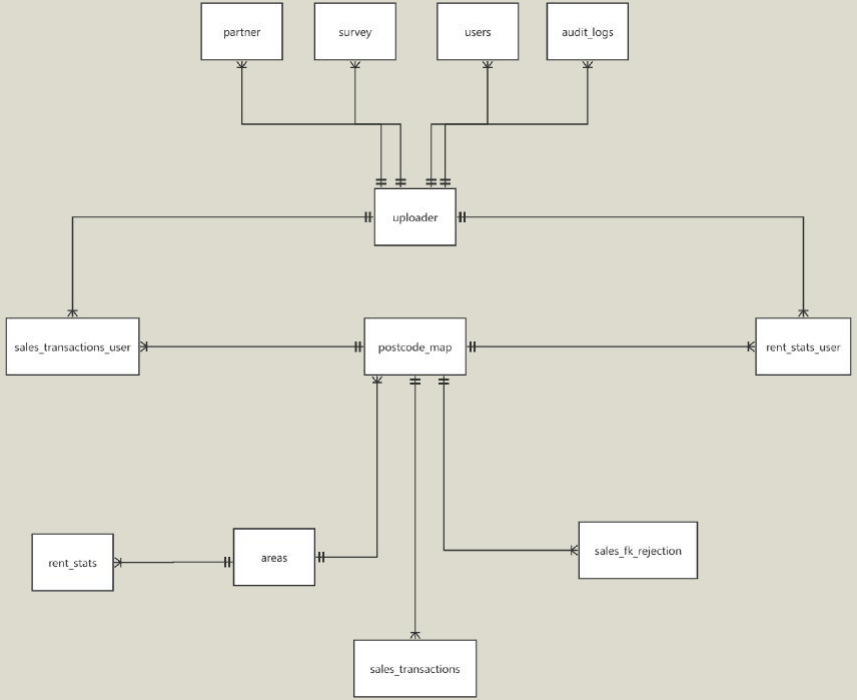
Data Sources and System Overview

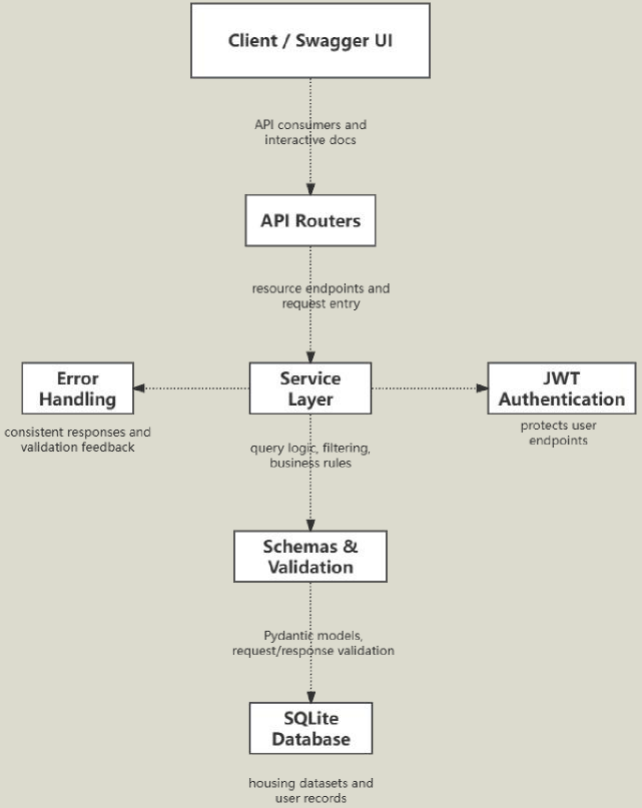
- Reference data: areas, postcode mapping
- Official datasets: rent statistics, sales transactions
- User data: user-submitted rental/sales records



Designed as a database-backed REST API with clear separation between data ingestion, service logic, and API routing.

Database And Architecture Design





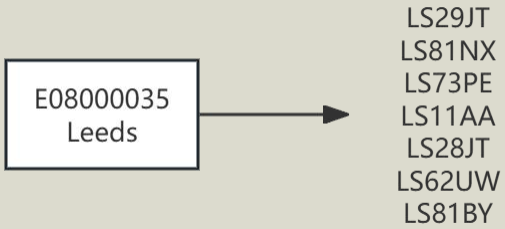
Shirui Zhao

2026-03-12

API Design Highlights

A Reference Geography

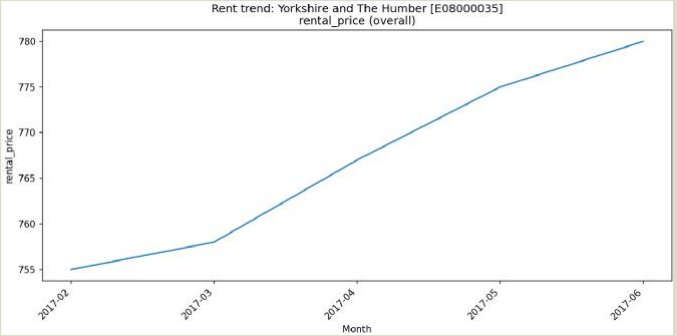
- GET /AREAS
- GET /AREAS/{AREA_CODE}
- GET /POSTCODE_MAP/{POSTCODE}



Reference lookup for areas and postcode mapping

B Official Statistics

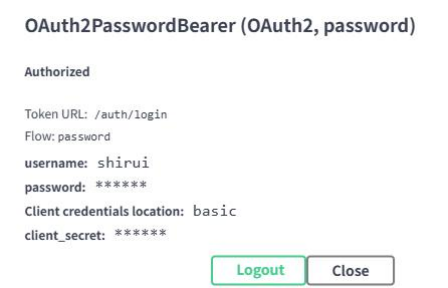
- Rent statistics by area and period
- Sales transaction queries and summaries
- Read-oriented access to official datasets



/rent_stats_official/areas/{area_code}/rent-trend.png

C User Records & Auth

- POST /register
- POST /login
- Protected CRUD for user records

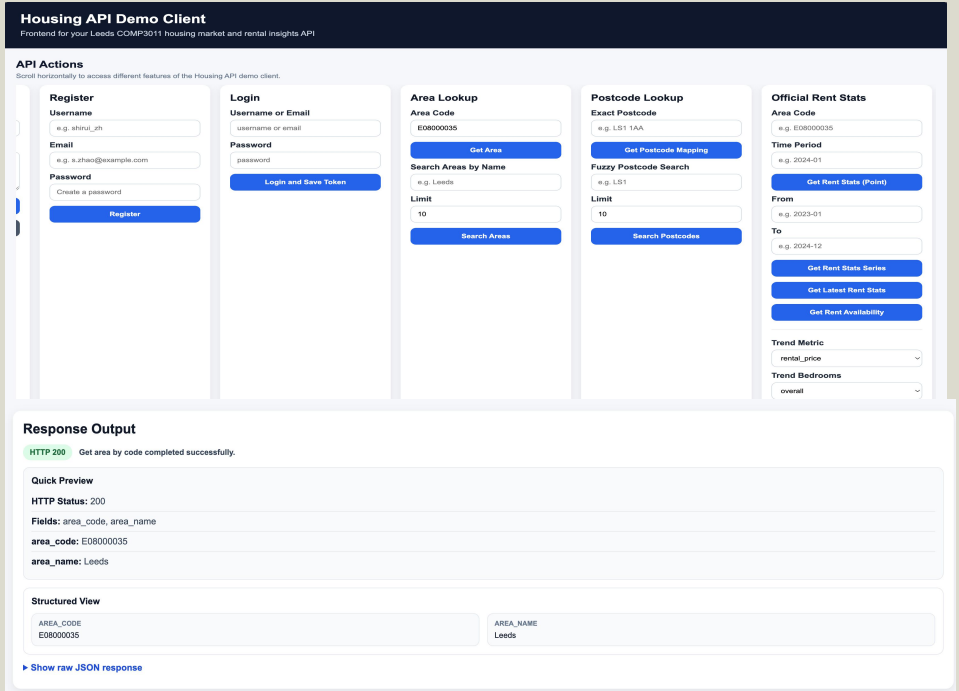


Authenticated operations for user-managed data

Shirui Zhao

2026-03-12

Frontend Interface



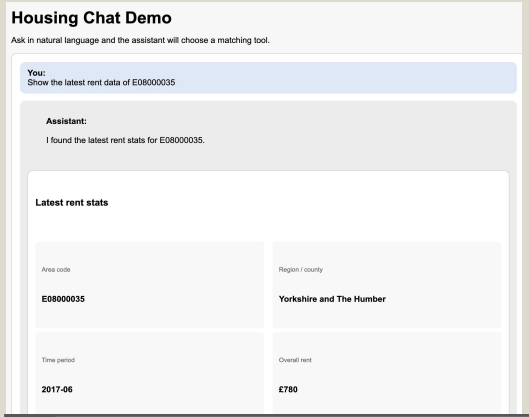
Thin client layer: presentation logic separated from backend service logic

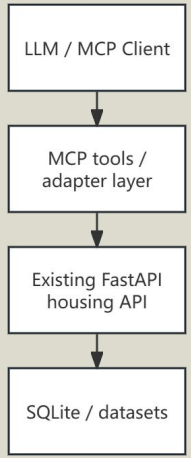
Key Points

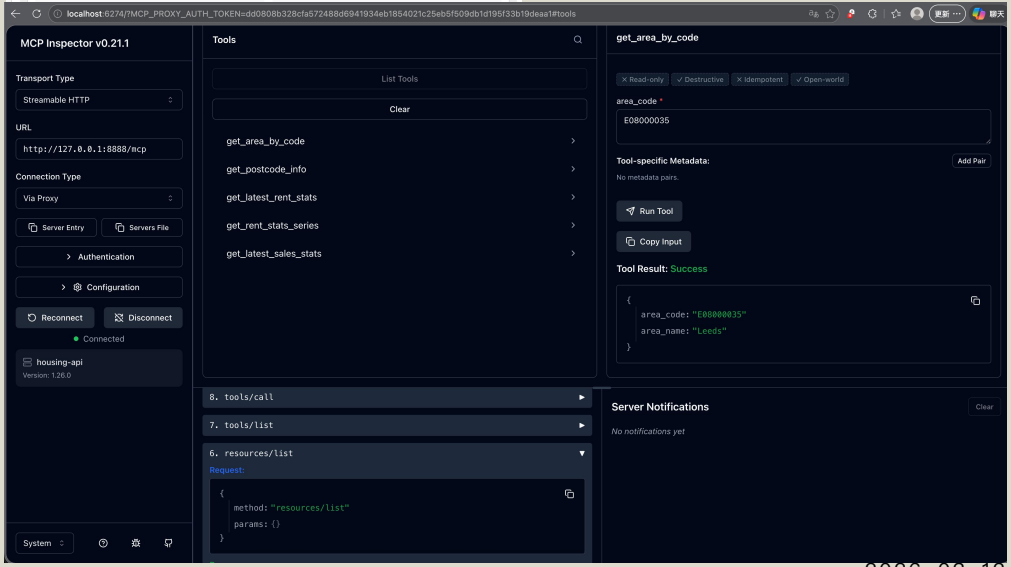
- Lightweight web interface built on top of the same FastAPI backend
- Supports human-centred exploration beyond raw Swagger/API calls
- Demonstrates selected workflows such as lookup, analytics, and user interaction

MCP Integration

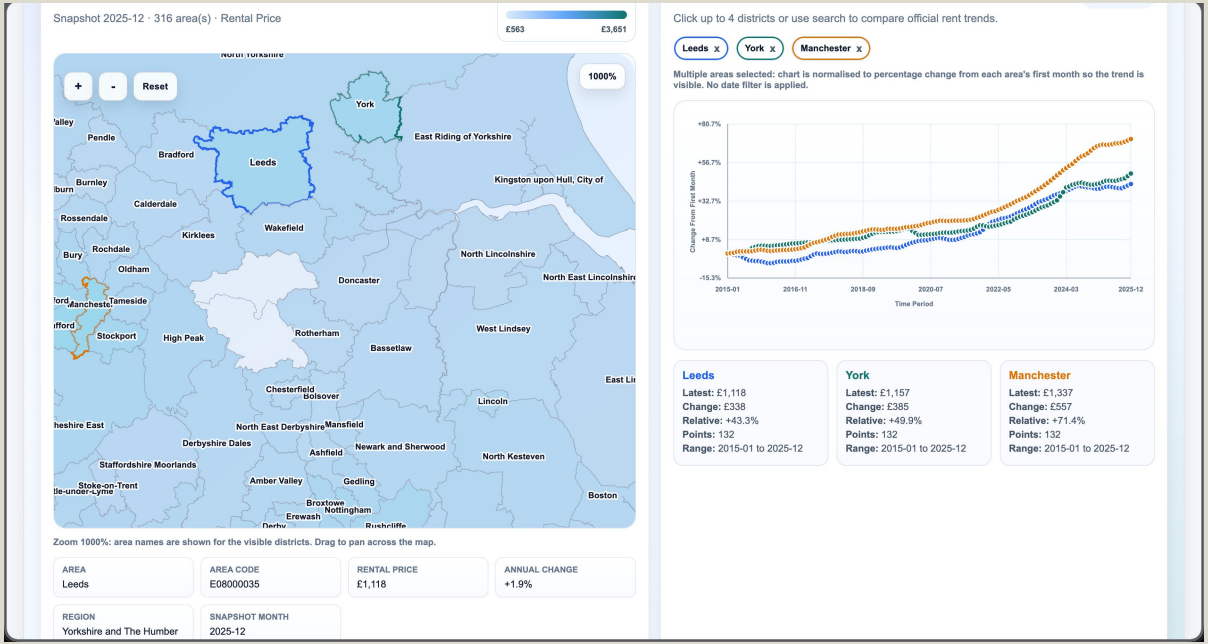
- Exposes selected housing-data capabilities through a tool-oriented interface
- Reuses existing backend logic rather than creating a separate service stack
- Shows extensibility beyond conventional REST consumption







Geographic Rent Explorer



- Loads historical rent data by area and time period
- Supports area selection and side-by-side comparison
- Improves interpretability for non-technical users

Testing, Deployment, and Deliverables

A Testing

- Endpoint tests with pytest
- Validation of success and error cases
- Coverage of query parameters and edge cases
- Verified expected status codes and responses

```
tests/test_area.py .......
tests/test_audit_logs.py ......
tests/test_auth.py .......
tests/test_postcode.py ......
tests/test_rent_official.py ................
tests/test_rent_user.py ......................
tests/test_sales_official.py .................
tests/test_sales_official_area.py ............
tests/test_sales_official_postcode.py ........
tests/test_sales_official_statistic.py .......
tests/test_sales_user.py .....................
```

B Deployment

- Deployed as a runnable web service
- Interactive API documentation available
- Submission version aligned with hosted demo
- CI/CD used for deployment validation

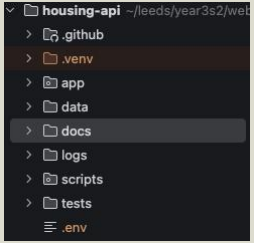
← CI

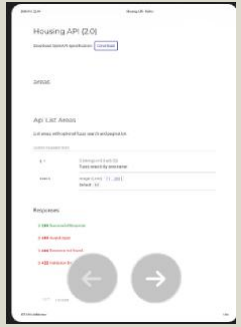
docs: update readme file #14

```
==> Your service is live 🎉
z855]  2026-03-10 01:40:00 | INFO | housing_api | request_id=6a3
z855]  INFO:     34.83.77.94:0 - "GET / HTTP/1.1" 404 Not Found
==>
==> ////////////////////////////////////////////////////////
==>
==> Available at your primary URL https://housing-api-p0j
```

C Deliverables

- Public GitHub repository with commit history
- API documentation and technical report
- Source code, database setup, and evidence of development
- Presentation and submission-ready materials



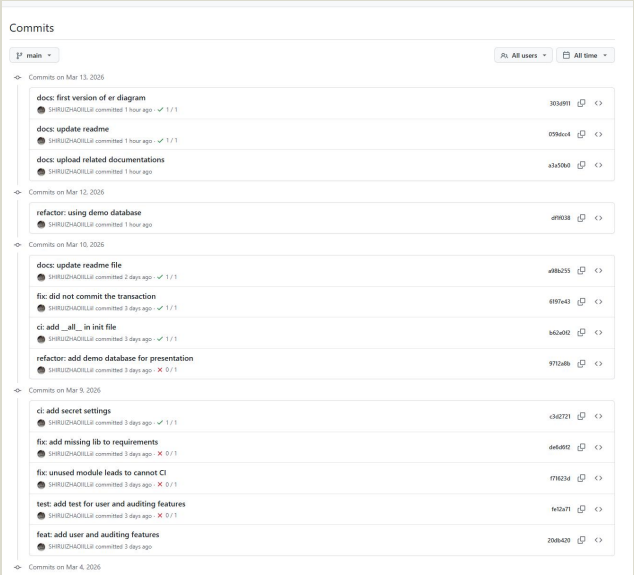


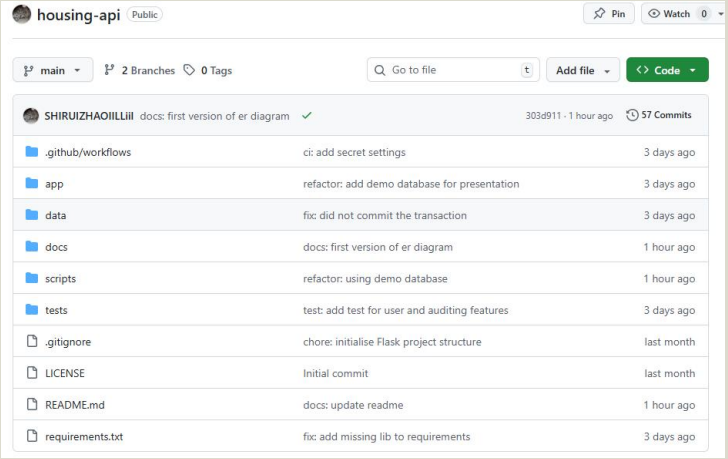
Shirui Zhao

2026-03-12

Version Control Practices and Commit History

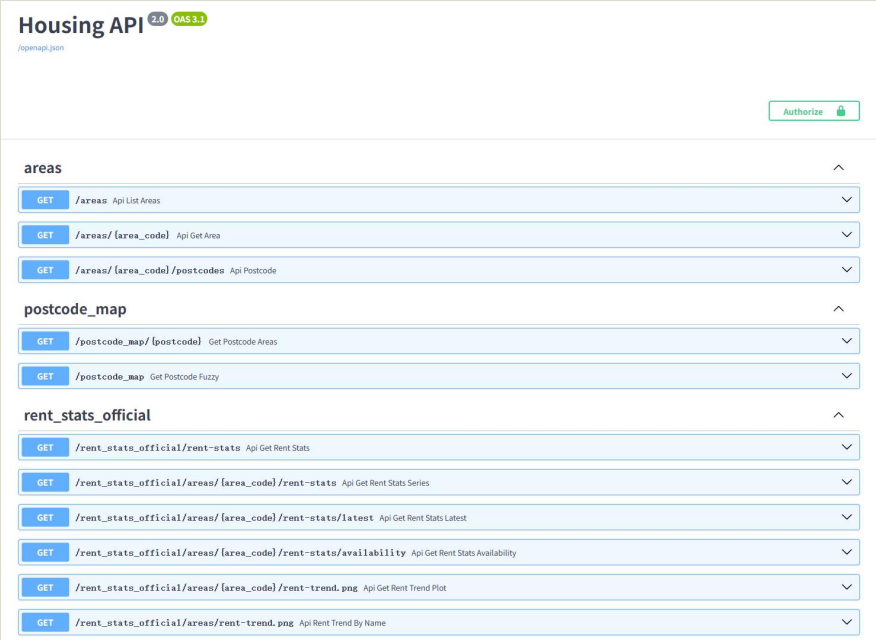
- Public GitHub repository used throughout development.
- Incremental commits documented database setup, API implementation, testing, and deployment changes.
- Commit history reflects staged development rather than one-time upload.

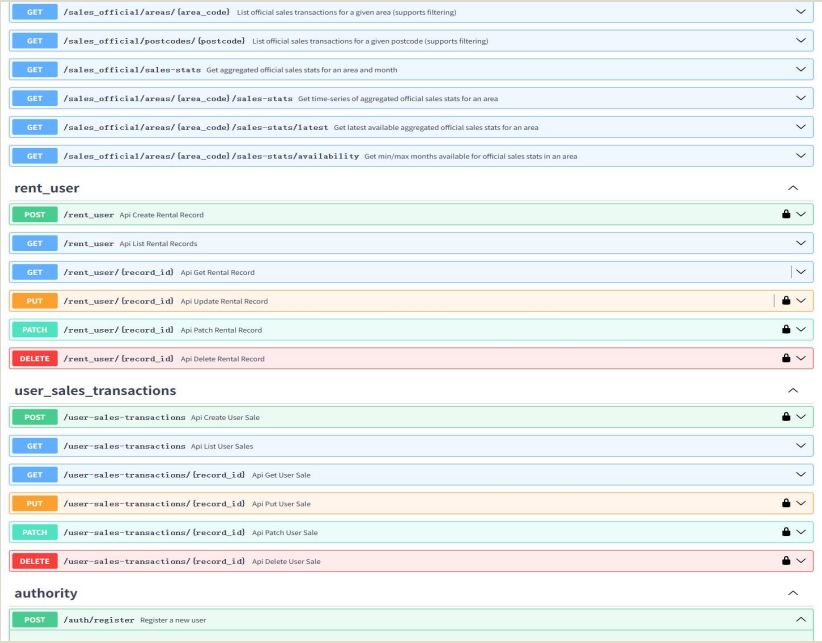




API Documentation Overview

- FastAPI auto-generated OpenAPI / Swagger documentation
- documents routes, request models, responses, authentication flows
- reduces drift between implementation and documentation





Technical Report Highlights

- Architecture: layered FastAPI design with routers, services, and database separation
- Design choice: official datasets separated from user-managed editable records
- Technology stack: FastAPI + SQLite chosen for portability, schema-first design, and coursework practicality
- Lessons learned: testing improved not just correctness, but API contract clarity

Project Links

- GitHub Repository: https://github.com/SHIRUIZHAOIILLiil/housing-api
- API Documentation PDF: https://github.com/SHIRUIZHAOIILLiil/housing-api/tree/main/docs/Housing-API.pdf
- Technical Report PDF: https://github.com/SHIRUIZHAOIILLiil/housing-api/tree/main/docs/Housing-API-technical-report.pdf
- Presentation Slides: https://github.com/SHIRUIZHAOIILLiil/housing-api/tree/main/docs/Housing-API.pptx

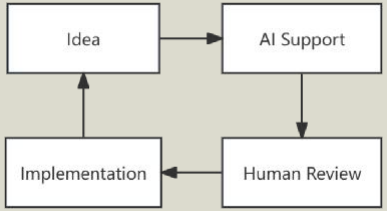
Reflection, GenAI Usage, and Future Work

A Reflection

- Learned practical REST API design and data modelling
- Improved testing, debugging, and project structuring skills
- Better understood how to turn raw datasets into usable services

B GenAI Usage

- Used GenAI for planning, debugging, and refining design ideas
- Outputs were reviewed, adapted, and validated before use
- Usage was documented in the report with supporting evidence



C Future Work

- Add richer analytics and comparison endpoints
- Introduce role-based authorization and stronger security controls
- Move to cloud database and more production-ready deployment

Shirui Zhao

That is the end. Thank you